DSA Lecture 10: Binary Search Trees (BSTs)

1. Introduction to Trees

Before diving into BSTs, recall the basic structure of a **Tree** in computer science.

- **Definition:** A tree is a non-linear, hierarchical data structure that consists of nodes connected by edges.
- **Key Terminology:**
  - **Root:** The topmost node of the tree.
  - **Child/Parent:** Nodes directly below a node are its children; the node above is the parent.
  - **Leaf Node:** A node with no children.
  - **Subtree:** A tree consisting of an arbitrary node of the original tree and all its descendants.

2. Binary Tree

A **Binary Tree** is a tree data structure in which each node has at most two children, typically referred to as the **left child** and the **right child**.

3. Binary Search Tree (BST)

A Binary Search Tree is a special type of Binary Tree that maintains a specific ordering property to facilitate efficient searching and retrieval.

The BST Property (Key Invariant)

For every node in a BST:

1. The value of all nodes in its **left subtree** must be **less than** the node's value.
2. The value of all nodes in its **right subtree** must be **greater than** the node's value.
3. Both the left and right subtrees must also be BSTs.

---

4. Operations on BSTs

The fundamental operations on a BST are searching, insertion, and deletion.

A. Search (or Lookup)

The search operation leverages the BST property to efficiently find a node with a specific key.

- **Algorithm:**
  1. Start at the **root** of the tree.
  2. Compare the **target key** with the current node's value.
  3. If they match, the node is found: **Return true**.
  4. If the target key is **less than** the node's value, proceed to search the **left subtree**.
  5. If the target key is **greater than** the node's value, proceed to search the **right subtree**.

6. If the current node is NULL (meaning you've reached an empty link), the key is not in the tree: **Return false**.

- **Time Complexity:**
  - **Best/Average Case:** O(log n) (Proportional to the height of the tree, h, which is approx $\log_2 n$ for a balanced tree).
  - **Worst Case:** O(n) (Occurs when the tree is **skewed** or degenerates into a linked list).

B. Insertion

A new node is always inserted as a **leaf node** to maintain the BST property.

- **Algorithm:**
  1. Start at the **root**.
  2. Perform a search for the key to be inserted.
  3. If the key already exists, typically do nothing (or update data, depending on requirements).
  4. If the key does not exist, the search will terminate at a NULL link, which is where the new node should be attached.
  5. Traverse down: If the key is **less than** the current node, go left. If **greater**, go right.
  6. When the insertion point is found, create the new node and link it to its parent.
- **Time Complexity:** Same as search: **Best/Average Case: O(log n), Worst Case: O(n)**.

C. Deletion

Deletion is the most complex operation, as simply removing a node can violate the BST property. There are three main cases for deleting a node N:

1. **Case 1: N is a Leaf Node (0 children).**
   - Simply remove N. Set the link from its parent to NULL.
2. **Case 2: N has 1 Child.**
   - Bypass N. Link N's single child directly to N's parent.
3. **Case 3: N has 2 Children.**
   - This is the critical case. The goal is to replace N with a node that maintains the BST property.
   - **Replacement Strategy:** Find the **inorder successor** (the smallest value in N's right subtree) or the **inorder predecessor** (the largest value in N's left subtree).
   - **Common Method (Inorder Successor):**
     - Find the **minimum element** in the right subtree of N. Let's call it S.
     - Copy the value of S into N.
     - Recursively delete S from N's right subtree (Note: S will have at most one child—a right child—making its deletion easy, falling under Case 1 or 2).

- **Time Complexity:** Same as search: **Best/Average Case: O(log n), Worst Case: O(n)**.

---

5. Traversals (Pre-requisite Knowledge)

Traversal methods are ways to visit every node in the tree exactly once. In BSTs, the **Inorder** traversal is particularly important.

Traversal	Order of Visit	Purpose
Inorder	Left -> Node -> Right	**Prints the elements in a sorted (ascending) order.**
Preorder	Node -> Left -> Right	Used for copying the tree structure.
Postorder	Left -> Right -> Node	Used for deleting the tree (since the root is deleted last).

---

6. BST Efficiency and Limitations

A. Complexity Summary (Height h)

Operation	Best/Average Case	Worst Case (Skewed Tree)
Search	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)

B. The Skewed Tree Problem

The primary limitation of a standard BST is that its performance heavily depends on the **order of insertion**.

- If elements are inserted in a **sorted or reverse-sorted order** (e.g., 1, 2, 3, 4, 5), the tree degenerates into a linear structure, essentially a **linked list**.
- In this worst-case scenario, the height h = n, and all operations degrade to O(n).

C. Solution: Self-Balancing BSTs

To guarantee the O(log n) performance for all operations, we use **Self-Balancing Binary Search Trees**. These structures automatically perform rotations or color changes during insertion and deletion to maintain a maximum height of O(log n).

- **Key Examples:**
    - **AVL Trees:** Maintain a *Balance Factor* (height difference of left and right subtrees is less equal 1).
    - **Red-Black Trees:** Maintain balance using color properties (used widely in programming language libraries, like std::map in C++ and TreeMap in Java).

7. Code

```
#include <iostream>
using namespace std;

// Define a BST Node
struct Node {
    int data;
    Node* left;
    Node* right;
};

// a) Creation: Create a new node
Node* createNode(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->left = nullptr;
    newNode->right = nullptr;
    return newNode;
}

// b) Insertion
Node* insert(Node* root, int value) {
    if (root == nullptr)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

// c) Find Minimum
Node* findMin(Node* root) {
    while (root != nullptr && root->left != nullptr)
        root = root->left;
    return root;
}

// d) Find Maximum
Node* findMax(Node* root) {
```

```
    while (root != nullptr && root->right != nullptr)
        root = root->right;
    return root;
}

// e) Delete Element
Node* deleteNode(Node* root, int key) {
    if (root == nullptr)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Node with one or no child
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Node with two children
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// f) Post-order Traversal
void postOrder(Node* root) {
    if (root == nullptr) return;
    postOrder(root->left);
    postOrder(root->right);
    cout << root->data << " ";
}
```

```
// g) Pre-order Traversal
void preOrder(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
}

// h) In-order Traversal
void inOrder(Node* root) {
    if (root == nullptr) return;
    inOrder(root->left);
    cout << root->data << " ";
    inOrder(root->right);
}

// Main function
int main() {
    Node* root = nullptr;

    int elements[] = {50, 30, 20, 40, 70, 60, 80};
    for (int i = 0; i < 7; i++) {
        root = insert(root, elements[i]);
    }

    cout << "In-order traversal: ";
    inOrder(root);
    cout << endl;

    cout << "Pre-order traversal: ";
    preOrder(root);
    cout << endl;

    cout << "Post-order traversal: ";
    postOrder(root);
    cout << endl;

    cout << "Minimum element: " << findMin(root)->data << endl;
    cout << "Maximum element: " << findMax(root)->data << endl;

    root = deleteNode(root, 20);
    cout << "After deleting 20, In-order: ";
    inOrder(root);
```

```
    cout << endl;

    root = deleteNode(root, 30);
    cout << "After deleting 30, In-order: ";
    inOrder(root);
    cout << endl;

    root = deleteNode(root, 50);
    cout << "After deleting 50, In-order: ";
    inOrder(root);
    cout << endl;

    return 0;
}
```